



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO  
FACULTAD DE INGENIERÍA  
DIVISIÓN DE INGENIERÍA ELÉCTRICA  
INGENIERÍA EN COMPUTACIÓN  
LABORATORIO DE COMPUTACIÓN GRÁFICA e  
INTERACCIÓN HUMANO COMPUTADORA



**EJERCICIOS DE CLASE N° 03**

**NOMBRE COMPLETO: Gomez Aceves Abraham Agustin**

**N° de Cuenta: 320093290**

**GRUPO DE LABORATORIO: 03**

**GRUPO DE TEORÍA: 05**

**SEMESTRE 2027-1**

**FECHA DE ENTREGA LÍMITE: 08 – septiembre - 2026**

**CALIFICACIÓN: \_\_\_\_\_**

## EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa
2. Problemas presentados. Listar si surgieron problemas a la hora de ejecutar el código
3. Conclusión:
  - a. Los ejercicios de la clase: Complejidad, explicación
  - b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias.

## Actividades realizadas y problemas encontrados

Se usará la proyección en perspectiva. Instanciar primitivas geométricas para recrear la figura central de la práctica pasada en 3D no importa que internamente se encimen los cubos. Cada pirámide de cada esquina es de un color diferente de acuerdo a los colores de las 4 pirámides de la clase pasada. Se requiere que exista el plano de piso negro.

Codigo

```
struct Instance
{
    int meshIndex;

    glm::mat4 transform; //transform final de la pieza (cara + posición + rotación +
escala)
    glm::vec3 color;
};
```

```

vector<Instance> figuralInstances;

void AddInstanceMatrix(int meshIndex, const glm::mat4& transform, glm::vec3
color)
{
    Instance inst;
    inst.meshIndex = meshIndex;
    inst.transform = transform;
    inst.color = color;
    figuralInstances.push_back(inst);
}

glm::mat4 PiezaLocal(glm::vec3 posPlano, glm::vec3 scale, float angleXDeg, float
angleZDeg)
{
    glm::mat4 m(1.0f);
    m = glm::translate(m, posPlano);
    m = glm::rotate(m, glm::radians(angleZDeg), glm::vec3(0.0f, 0.0f, 1.0f));
    m = glm::rotate(m, glm::radians(angleXDeg), glm::vec3(1.0f, 0.0f, 0.0f));
    m = glm::scale(m, scale);
    return m;
}

void AddInstanceFull(const glm::mat4& cara, int meshIndex, glm::vec3 posPlano,
glm::vec3 scale,
    float angleXDeg, float angleZDeg, glm::vec3 color)
{
    AddInstanceMatrix(meshIndex, cara * PiezaLocal(posPlano, scale, angleXDeg,
angleZDeg), color);
}

```

```
}
```

```
void AddInstance(const glm::mat4& cara, int meshIndex, glm::vec3 posPlano,  
glm::vec3 scale, float angleZDeg, glm::vec3 color)
```

```
{
```

```
    AddInstanceFull(cara, meshIndex, posPlano, scale, 0.0f, angleZDeg, color);
```

```
}
```

Por que se hizo de esta forma

El archivo base dibuja UNA sola figura por cuadro, con UNA sola matriz de modelo. El diseño final tiene más de 80 piezas (marco, esquinas, círculo, centro, repetidas en 6 caras). Crear una malla (Mesh) distinta para cada una habría multiplicado por 80 el uso de memoria y el código repetido. En vez de eso, cada pieza solo guarda qué malla reutiliza, su matriz final y su color; el ciclo de dibujo (dentro de main) recorre la lista una sola vez con un for, sin importar cuántas piezas existan.

Problemas que hubo que resolver

Sin este sistema, repetir el mismo panel decorativo en las 6 caras del cubo habría significado escribir manualmente cada pieza 6 veces (una por cara), con altísimo riesgo de errores de copiado. Con AddInstance, la función que arma el panel se escribió una sola vez (ConstruirPanelEnCara) y se le pasa la orientación de cada cara.

Codigo

```
glm::mat4 TransformCara(glm::vec3 normal, float anguloYDeg, float anguloXDeg)
```

```
{
```

```
    glm::mat4 m(1.0f);
```

```
    m = glm::translate(m, normal * MITAD_CUBO);
```

```
    m = glm::rotate(m, glm::radians(anguloYDeg), glm::vec3(0.0f, 1.0f, 0.0f));
```

```

    m = glm::rotate(m, glm::radians(anguloXDeg), glm::vec3(1.0f, 0.0f, 0.0f));
    return m;
}

const glm::mat4 CARA_FRENTE = TransformCara(glm::vec3(0.0f, 0.0f, 1.0f),
0.0f, 0.0f);

const glm::mat4 CARA_ATRAS = TransformCara(glm::vec3(0.0f, 0.0f, -1.0f),
180.0f, 0.0f);

const glm::mat4 CARA_DERECHA = TransformCara(glm::vec3(1.0f, 0.0f, 0.0f),
90.0f, 0.0f);

const glm::mat4 CARA_IZQUIERDA = TransformCara(glm::vec3(-1.0f, 0.0f, 0.0f),
-90.0f, 0.0f);

const glm::mat4 CARA_ARRIBA = TransformCara(glm::vec3(0.0f, 1.0f, 0.0f), 0.0f,
-90.0f);

const glm::mat4 CARA_ABAJO = TransformCara(glm::vec3(0.0f, -1.0f, 0.0f), 0.0f,
90.0f);

```

Por que se hizo de esta forma

El diseño del panel (marco + esquinas + rombo + centro) se piensa en un plano 2D local: (x, y) es la posición dentro de esa cara y una tercera coordenada "d" es qué tanto sobresale la pieza hacia afuera del cubo. TransformCara calcula, para cada una de las 6 caras, la matriz que gira ese plano local hasta que su normal apunte en la dirección correcta, y lo aleja MITAD\_CUBO unidades del centro. Así, ConstruirPanelEnCara(cara) se escribe una sola vez y se reutiliza con las 6 matrices.

Problemas que hubo que resolver

En la primera versión de esta práctica, todo el panel decorativo (marco, esquinas, rombo, círculo) solo aparecía en la cara frontal del cubo, las otras 5 caras se veían lisas. Sin este sistema de transformación por cara, hubiera sido necesario reescribir manualmente el panel 6 veces con coordenadas distintas para cada una.

## Codigo

```
void CrearTrianguloEsquina()
{
    float b = MARCO_BORDE_INTERIOR / MITAD_CUBO;

    unsigned int trianguloesquina_indices[] = {
        0,1,2, 0,2,1,
        3,5,4, 3,4,5,
        0,1,4, 0,4,1, 0,4,3, 0,3,4,
        1,2,5, 1,5,2, 1,5,4, 1,4,5,
        2,0,3, 2,3,0, 2,3,5, 2,5,3
    };

    GLfloat trianguloesquina_vertices[] = {
        1.0f, 1.0f, 0.5f,
        b, 1.0f, 0.5f,
        1.0f, b, 0.5f,
        1.0f, 1.0f, -0.5f,
        b, 1.0f, -0.5f,
        1.0f, b, -0.5f
    };

    Mesh* trianguloEsquina = new Mesh();
    trianguloEsquina->CreateMesh(trianguloesquina_vertices,
    trianguloesquina_indices, 18, 48);
    meshList.push_back(trianguloEsquina);
}
```

## Por que se hizo de esta forma

Cada cara del cubo, dibujada por separado, sí puede mostrar sus 4 colores exactos sin conflicto con las caras vecinas (a diferencia del intento 1). El triángulo se modeló para la esquina superior-derecha únicamente; las otras 3 esquinas se obtienen rotando la misma malla 90°, 180° y 270°.

## Problemas que hubo que resolver

Al verlo renderizado, la pieza se veía como una rayita, no como un triángulo relleno. Al principio la cara ancha del triángulo probablemente no se dibujaba por el sentido de sus índices (orden de los vértices). Se corrigió agregando cada triángulo por duplicado, con los dos sentidos de giro posibles (por eso el arreglo de índices de arriba tiene el doble de entradas de lo mínimo necesario).

Ademas el arreglo anterior no fue suficiente. La causa real era z-fighting: esta pieza y el marco gris (ConstruirMarcoExterior) ocupan exactamente el mismo espacio en la esquina y estaban a la misma profundidad ( $d = 0.05$  los dos), así que la tarjeta gráfica no podía decidir cuál dibujar encima de forma consistente. La solución fue separar la profundidad de esta pieza ( $d = 0.12$ ) de la del marco ( $d = 0.05$ ), para que siempre gane y se vea completa.

## Codigo

```
void CrearTrianguloDiamante()
{
    unsigned int indices[] = {
        0,1,2, 0,2,1,
        3,5,4, 3,4,5,
        0,1,4, 0,4,1, 0,4,3, 0,3,4,
        1,2,5, 1,5,2, 1,5,4, 1,4,5,
        2,0,3, 2,3,0, 2,3,5, 2,5,3
    };
    GLfloat vertices[] = {
        1.0f, 1.0f, 0.5f,
        0.0f, 1.0f, 0.5f,
        1.0f, 0.0f, 0.5f,
        1.0f, 1.0f, -0.5f,
```

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 0.0f, 1.0f, -0.5f, 1.0f, 0.0f, -0.5f }; Mesh* trianguloDiamante = new Mesh(); trianguloDiamante-&gt;CreateMesh(vertices, indices, 18, 48); meshList.push_back(trianguloDiamante); </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <p>Por que se hizo de esta forma</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <p>Sin ninguna pieza decorativa en el centro de la cara, el área que queda expuesta es simplemente la superficie plana del cubo (azul), y como el marco forma un cuadrado, esa área expuesta también se veía como un cuadrado.</p>                                                                                                                                                                                                                                                                                                                                                                                             |
| <p>Problemas que hubo que resolver</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <p>Se convertio el área azul cuadrada en un rombo sin perder el marco visible como cuadro completo. También se corrigió, en este mismo punto, un bug de la función ConstruirRombo (que dibujaba líneas dentro del rombo) tenía los ángulos de rotación mal calculados, las barras apuntaban hacia las esquinas en vez de seguir el borde del diamante. Era invisible porque esas barras eran azules sobre fondo azul, pero al agregar el corte gris quedaron expuestas como líneas cruzando el rombo. Se eliminaron esas llamadas en vez de recalcular los ángulos, porque la forma del rombo ya la daba esta nueva pieza.</p> |

|                                                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Codigo</p>                                                                                                                                                            |
| <pre> void CrearEsquinaPentagono() {     float b = MARCO_BORDE_INTERIOR / MITAD_CUBO;      unsigned int indices[] = {         2,3,4, 2,4,3,         2,4,0, 2,0,4, </pre> |



|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 2,0,1, 2,1,0, 7,8,9, 7,9,8, 7,9,5, 7,5,9, 7,5,6, 7,6,5, 0,1,6, 0,6,1, 0,6,5, 0,5,6, 1,2,7, 1,7,2, 1,7,6, 1,6,7, 2,3,8, 2,8,3, 2,8,7, 2,7,8, 3,4,9, 3,9,4, 3,9,8, 3,8,9, 4,0,5, 4,5,0, 4,5,9, 4,9,5 }; GLfloat vertices[] = {     b,  0.0f, 0.5f,     1.0f, 0.0f, 0.5f,     1.0f, 1.0f, 0.5f,     0.0f, 1.0f, 0.5f,     0.0f, b,  0.5f,     b,  0.0f, -0.5f,     1.0f, 0.0f, -0.5f,     1.0f, 1.0f, -0.5f,     0.0f, 1.0f, -0.5f,     0.0f, b,  -0.5f }; Mesh* esquinaPentagono = new Mesh(); esquinaPentagono-&gt;CreateMesh(vertices, indices, 30, 96); meshList.push_back(esquinaPentagono); } </pre> |
| <p>Por que se hizo de esta forma</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <p>Se modificaron lo triangulos de color de cada esquina hasta tocar directamente el</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |

rombo azul, cubriendo toda la esquina de una sola vez. Un pentágono de 5 vértices (la esquina real de la cara, los dos puntos donde el borde se junta con el marco, y las dos puntas del rombo) permite exactamente eso en una sola pieza, sin dejar ningún hueco. Los 5 vértices se dividen en 3 triángulos trazando diagonales desde el vértice de la esquina (triangulación en abanico).

Problemas que hubo que resolver

Esto significa que el marco gris ya no forma un cuadro completo con esquinas — solo se ve como 4 barras rectas a la mitad de cada borde, tapado en las esquinas por el color.

Codigo

```
CrearCubo();
CrearPiramideTriangular();
CrearCilindro(10, 1.0f);
CrearCono(25, 2.0f);
CrearPiramideCuadrangular();
CrearTrianguloEsquina();
CrearTrianguloDiamante();
CrearEsquinaPentagono();
CreateShaders();

ConstruirFiguraCompleta();
...
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -9.5f));
...
for (size_t idx = 0; idx < figuraInstances.size(); idx++)
{
```

```

Instance& inst = figuralInstances[idx];

model = glm::mat4(1.0f);

model = glm::translate(model, glm::vec3(0.0f, 0.0f, -9.5f));

    model = glm::rotate(model, glm::radians(mainWindow.getrotax()),
glm::vec3(1.0f, 0.0f, 0.0f));

    model = glm::rotate(model, glm::radians(mainWindow.getrotay()),
glm::vec3(0.0f, 1.0f, 0.0f));

    model = glm::rotate(model, glm::radians(mainWindow.getrotaz()),
glm::vec3(0.0f, 0.0f, 1.0f));

model = model * inst.transform;

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));

glUniform3fv(uniformColor, 1, glm::value_ptr(inst.color));

meshList[inst.meshIndex]->RenderMesh();
}

```

Por que se hizo de esta forma

El archivo base dibuja un solo mesh fijo (meshList[3]) con un solo color fijo, una vez por cuadro. La versión final necesita dibujar todas las piezas de figuralInstances, cada una con su propia malla, transform y color, por eso el cuerpo del ciclo cambió de una serie de llamadas sueltas a un for que arma la matriz de cámara/rotación global una vez por pieza y la combina con la matriz ya calculada de esa pieza (model \* inst.transform).

Problemas que hubo que resolver

El archivo base dibuja primitivas de tamaño unitario (aprox. 1 unidad). La figura final es un cubo de 3.6 unidades de lado con piezas que sobresalen un poco más si se dejaba la cámara a -4.0f, la figura quedaba demasiado cerca y se recortaba contra el plano de la cámara.

## Referencias

- de Berg, M., Cheong, O., van Kreveld, M., & Overmars, M. (2008). Computational geometry: Algorithms and applications (3rd ed.). Springer.
- de Vries, J. (s.f.). Depth testing. LearnOpenGL. <https://learnopengl.com/Advanced-OpenGL/Depth-testing>
- de Vries, J. (s.f.). Face culling. LearnOpenGL. <https://learnopengl.com/Advanced-OpenGL/Face-culling>
- de Vries, J. (s.f.). Transformations. LearnOpenGL. <https://learnopengl.com/Getting-started/Transformations>
- Hearn, D., Baker, M. P., & Carithers, W. R. (2011). Computer graphics with OpenGL (4th ed.). Prentice Hall.
- Khronos Group. (2024). glDrawElements [Referencia de la API]. OpenGL 4.5 Reference Pages. <https://registry.khronos.org/OpenGL-Refpages/gl4/html/glDrawElements.xhtml>
- Riccio, C. (2005–presente). OpenGL Mathematics (GLM): Manual y referencia. <https://github.com/g-truc/glm/blob/master/manual.md>